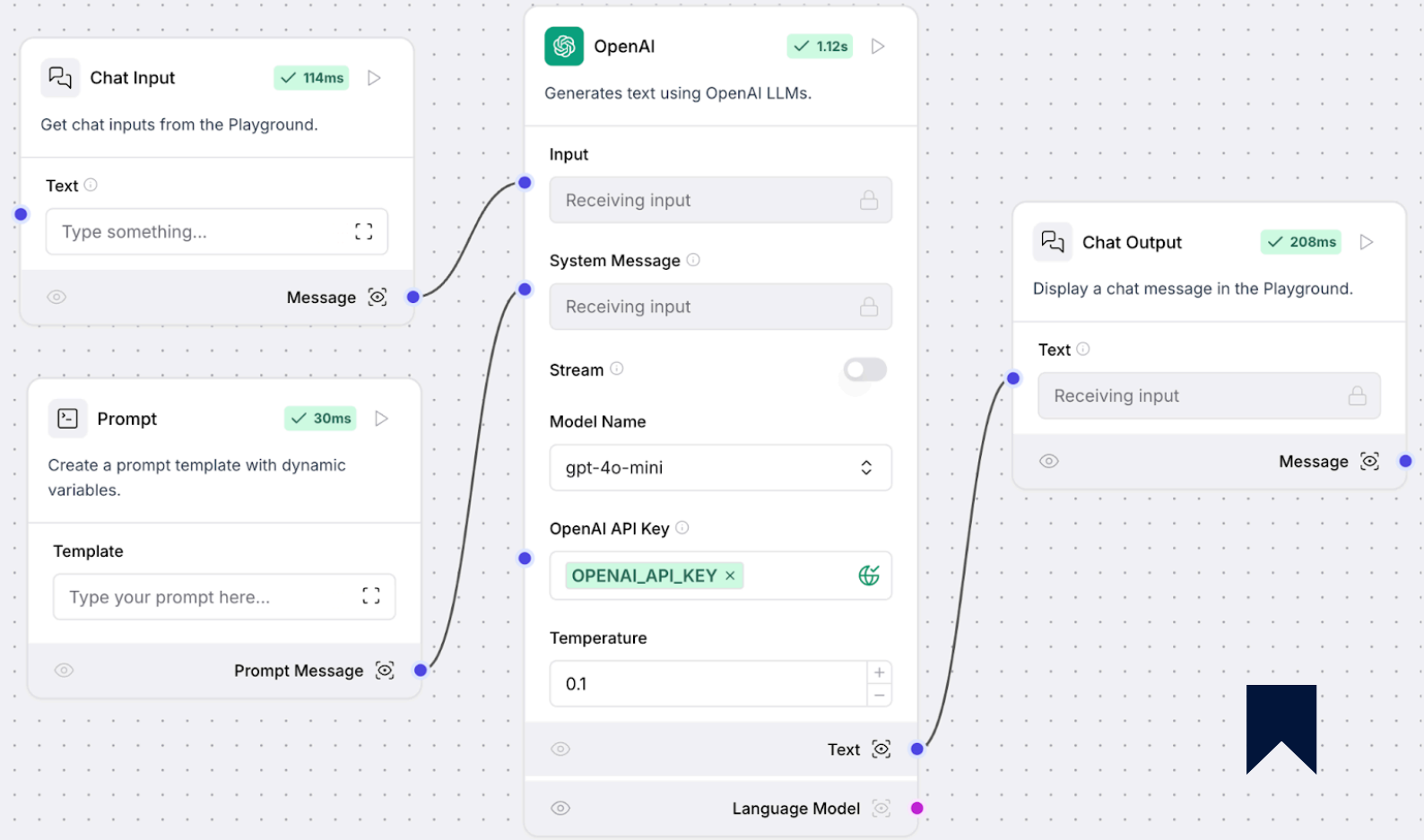
Langflow

HOW TO BUILD AN AI AGENT IN LANGFLOW?

From Setup to Deployment

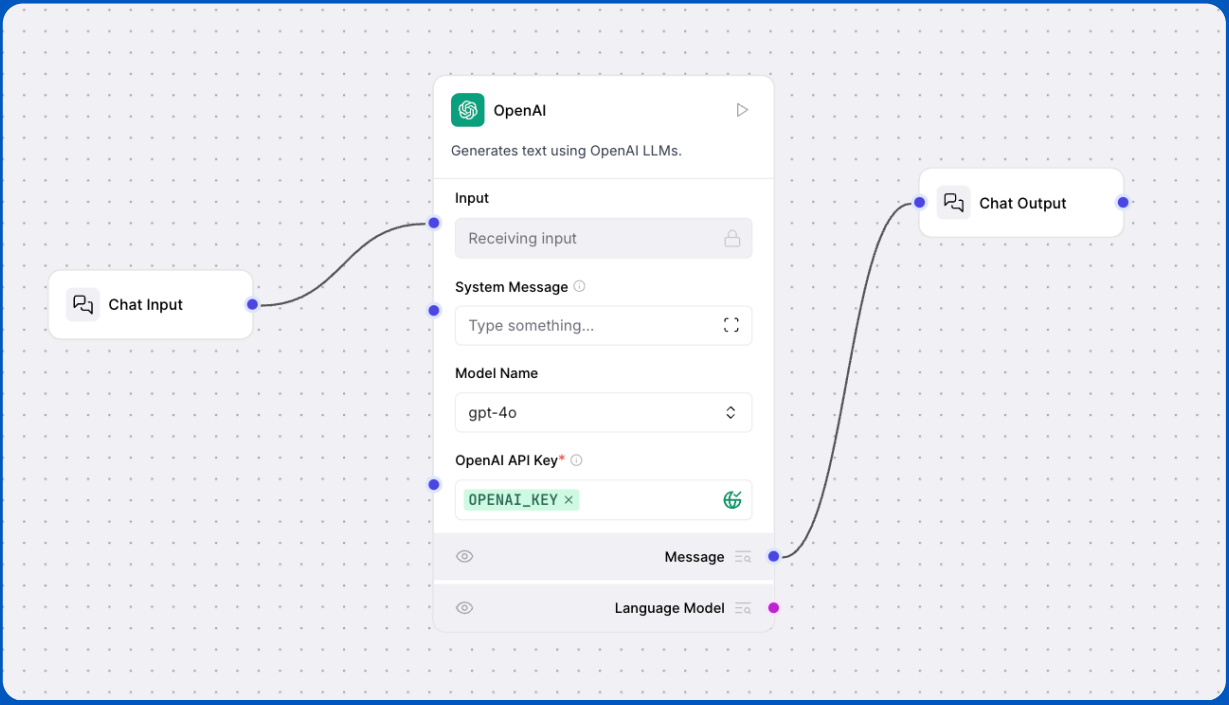
SLIDE TO EXPLORE



BASIC LLM COMPONENT (LANGFLOW)

Overview

This flow demonstrates a minimal, modular LLM agent pipeline in Langflow, designed to showcase the core building blocks of a conversational AI system. The flow is defined in the Basic LLM component.json file and visually represented in the diagram below.



FLOW STRUCTURE

The flow consists of three main components:

→ Chat Input

- Collects user input from the playground interface.
- Supports advanced options like sender type, session ID, file attachments, and message metadata (background color, icon, etc).
- Produces a Message object as output.

→ Model Component (OpenAI)

- Receives the Message from the Chat Input.
- Connects to OpenAI's LLMs (e.g., GPT-4o, GPT-3.5-turbo) using an API key.
- Configurable parameters: model name, temperature, max tokens, system message, streaming, and more.
- Returns a Message object containing the LLM's response.
- Instead of OpenAI component it can also be other providers present in Langflow.

FLOW STRUCTURE

→ **Chat Output**

- Takes the LLM's response and displays it in the playground.
- Handles formatting, sender metadata, and optional message storage.

HOW IT WORKS

→ **User Message:** The user enters a prompt in the Chat Input component.

→ **LLM Processing:** The message is sent to the OpenAI Model, which generates a response using the selected LLM.

→ **Display:** The response is passed to the Chat Output component, which renders it in the UI.

KEY FEATURES

→ **Modularity:** Each component is independent and reusable.

→ **Configurability:** Easily switch LLM providers or adjust parameters.

→ **Extensibility:** Add more nodes (e.g., memory, tools, RAG) to build advanced agents.

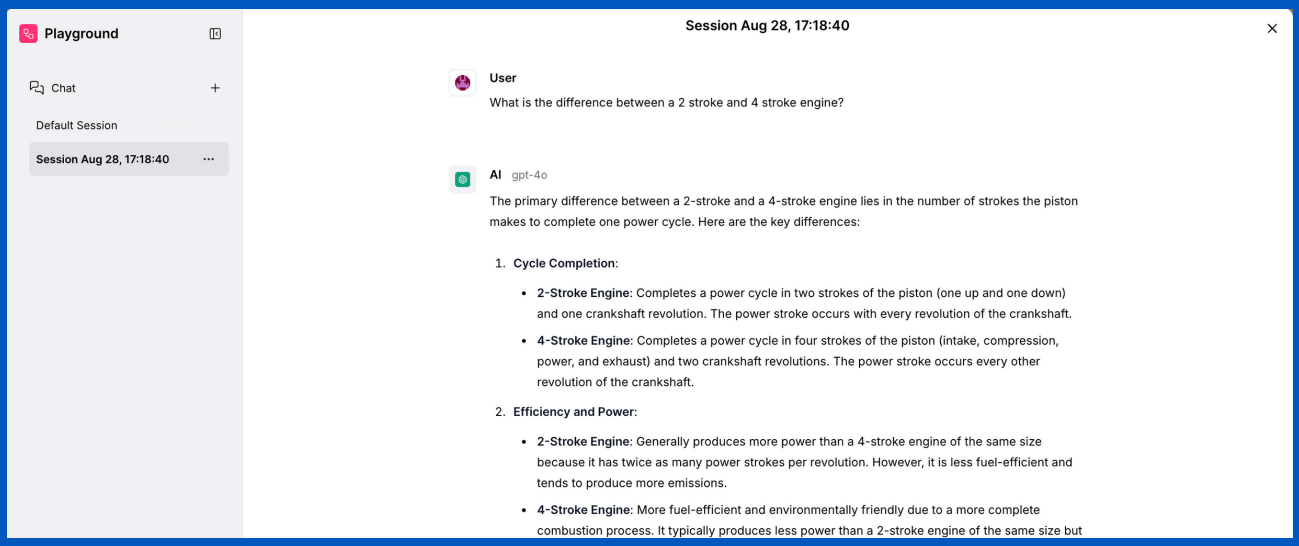
EXAMPLE USE CASE

This flow is ideal for:

- Prototyping simple chatbots
- Educational demonstrations of LLM agent architecture
- Serving as a template for more complex agentic workflows

OUTPUT

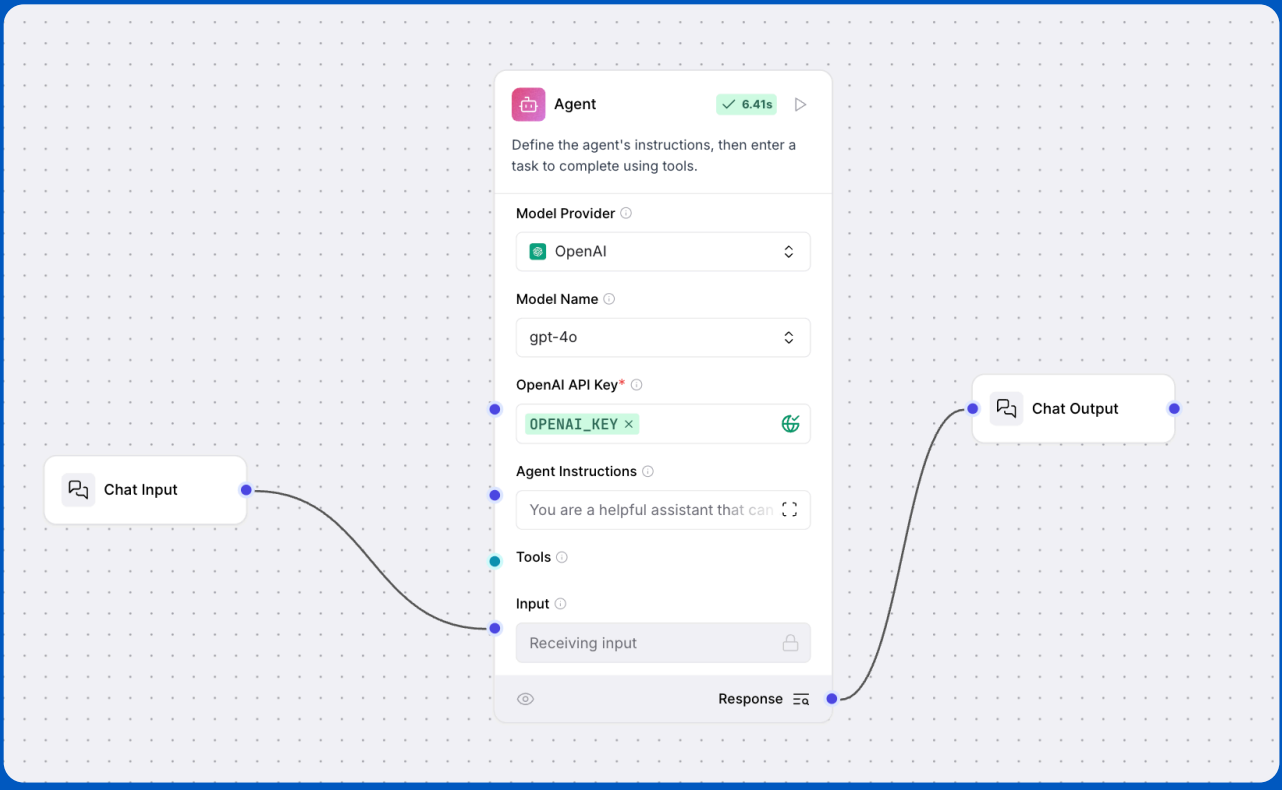
The output after trying it on the playground.



LLM WITH MEMORY (LANGFLOW)

This level demonstrates two approaches to building a chatbot that can remember previous conversation turns using Langflow:

1. Agent Component (with Built-in Memory)



LLM WITH MEMORY (LANGFLOW)

Overview

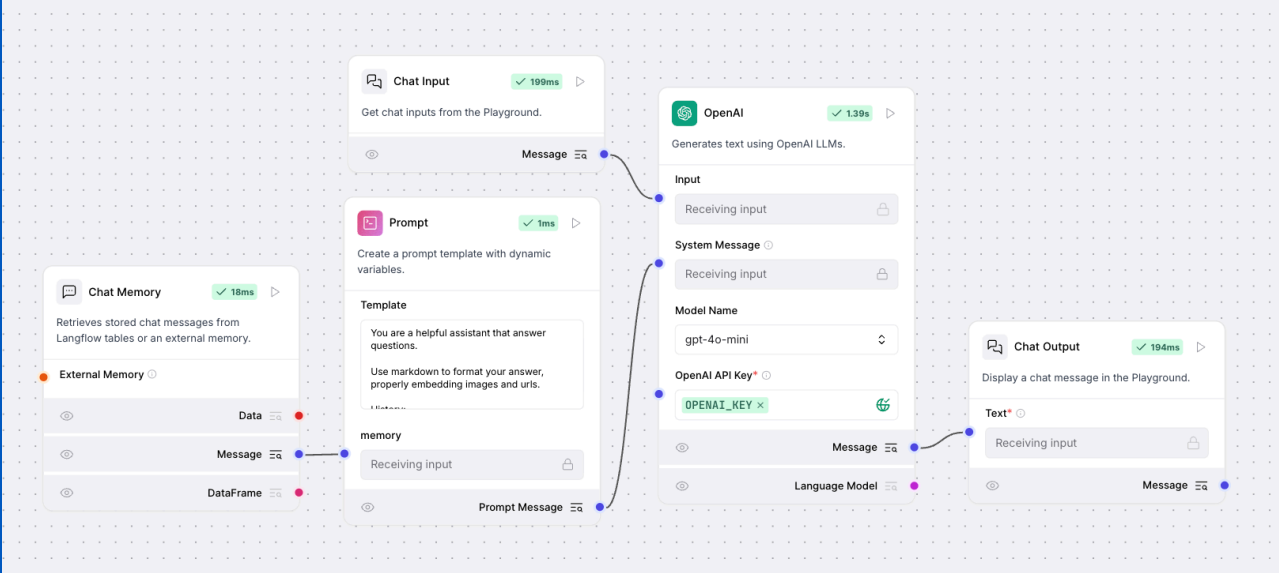
- Uses the Agent node, which internally manages conversation memory.
- The user message is passed from Chat Input to Agent, and the response is sent to Chat Output.
- No explicit memory node is required—the agent handles context tracking automatically.

Use Case:

- When you want a simple, plug-and-play agent that can remember previous messages without extra configuration.

LLM WITH MEMORY (LANGFLOW)

2. LLM with Explicit Memory Component

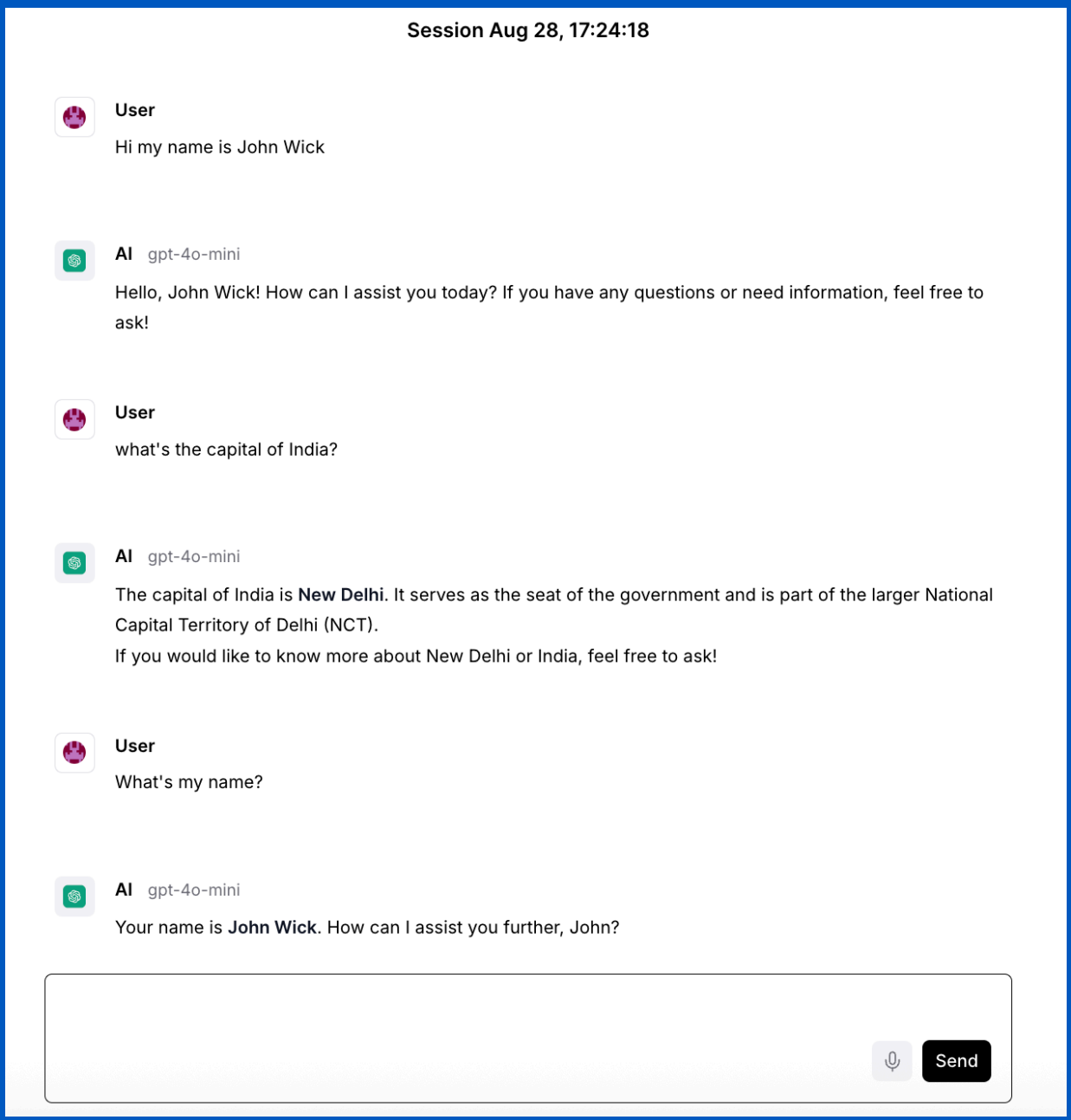


Overview

- Adds a Memory node to explicitly store and retrieve conversation history.
- The Memory node connects to a Prompt node, which injects the conversation history into the system prompt for the LLM.
- The OpenAI node receives both the user input and the memory-augmented prompt, then outputs to Chat Output.

OUTPUT

The output after trying it on the playground.

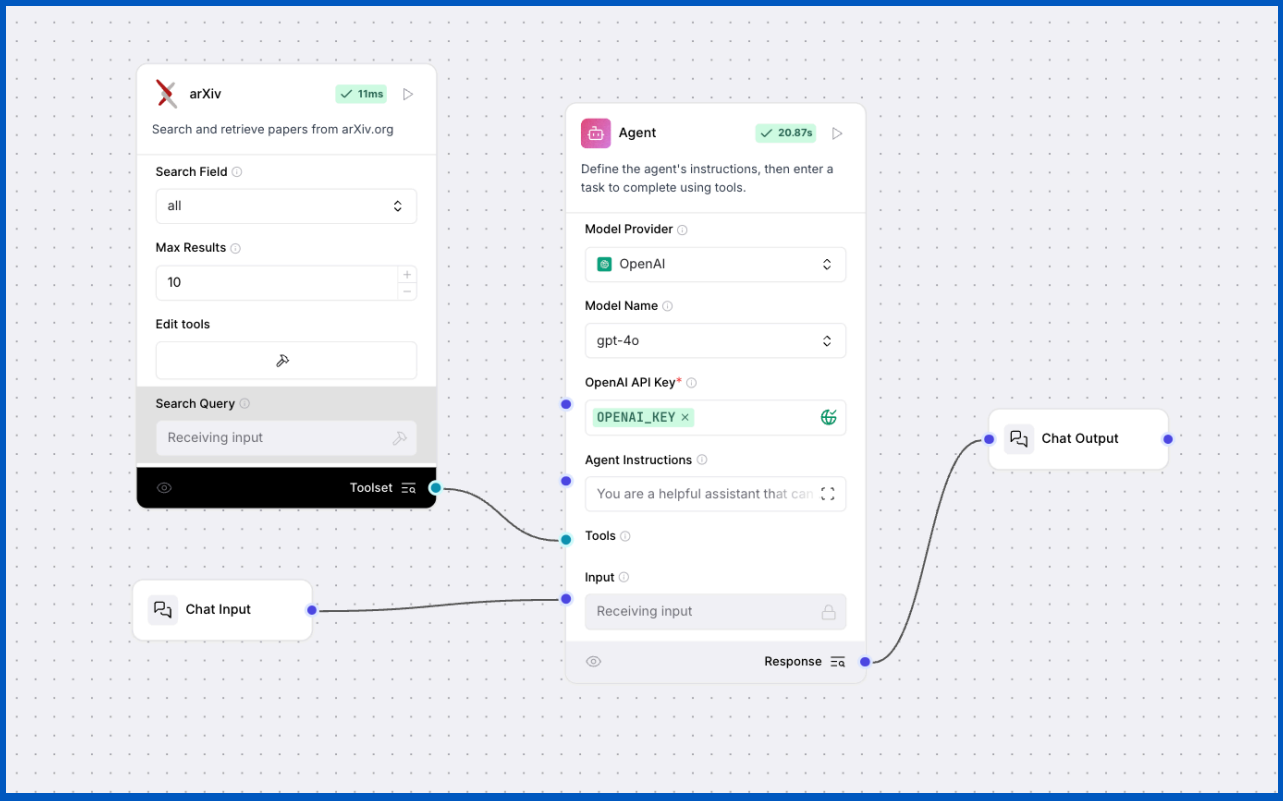


Summary

Both approaches enable persistent conversation context by making use of memory, which is an important factor when building Agentic AI systems.

AGENT WITH TOOL (LANGFLOW)

This level demonstrates how to build an agent that can use external tools to enhance its capabilities, using Langflow.



STRUCTURE

- **Chat Input:** Collects user messages.
- **Agent:** Receives user input and is configured with one or more tools (e.g., ArXiv search, web search, calculators, etc.).
- **Tool Node (e.g., ArXivComponent):** Provides the agent with access to an external capability. In this example, the agent can search academic papers via ArXiv.
- **Chat Output:** Displays the agent's response, which may include information retrieved using the tool.

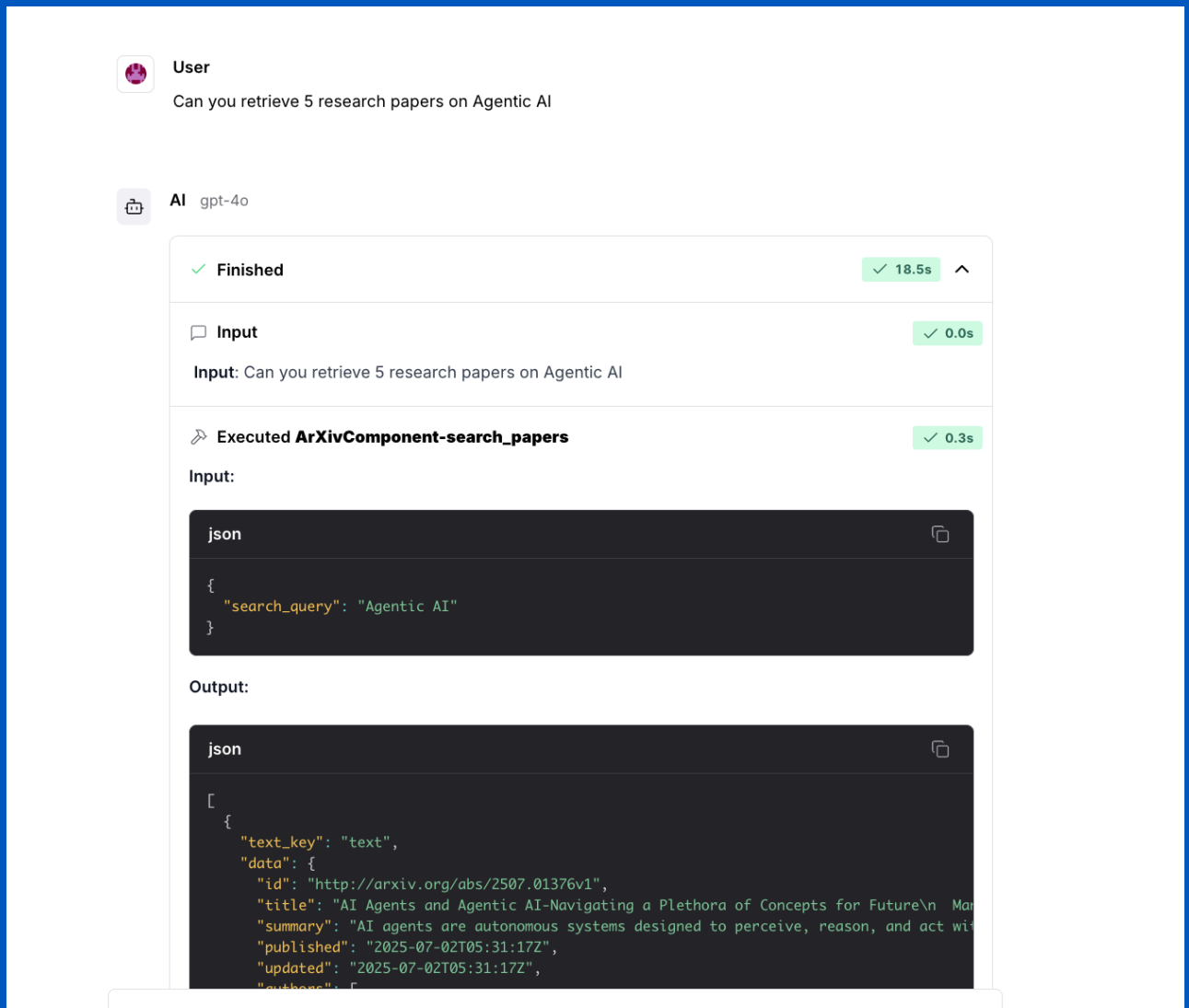
HOW IT WORKS

- The user enters a query in the Chat Input.
- The Agent determines if a tool is needed to answer the query.
- If required, the Agent invokes the tool (e.g., ArXiv search) and incorporates the result into its response.
- The final answer is shown in the Chat Output.

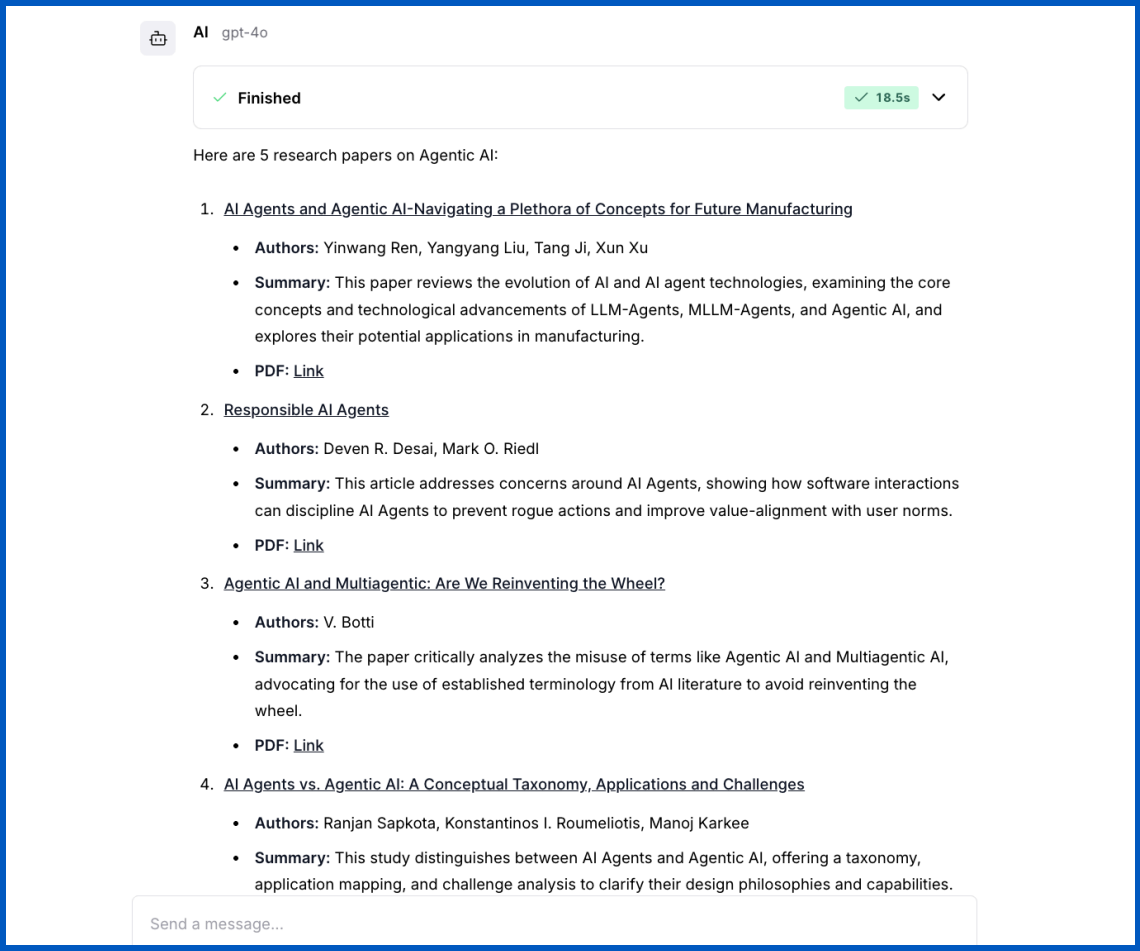
EXAMPLE USE CASES

- Academic research assistants (searching papers, summarizing findings)
- Agents that can access APIs, perform calculations, or retrieve real-time data
- Extending LLMs with custom or domain-specific tools

OUTPUT



OUTPUT



Key Points

- **Tool Integration:** Tools are passed to the agent as separate nodes and can be swapped or extended easily.
- **Agent Reasoning:** The agent decides when and how to use tools, making it more powerful than a plain LLM.
- **Modularity:** This pattern can be extended to multiple tools for more complex workflows.

RETRIEVAL-AUGMENTED GENERATION (RAG) PIPELINE WITH LANGFLOW

This level demonstrates how to build a Retrieval-Augmented Generation (RAG) pipeline using Langflow.

The solution is split into two main flows:

- **Ingestion Pipeline:** Loads and indexes content into a vector store (Astra DB).
- **Retrieval Pipeline:** Answers user questions by retrieving relevant context from the vector store and passing it to an LLM.

1. INGESTION PIPELINE

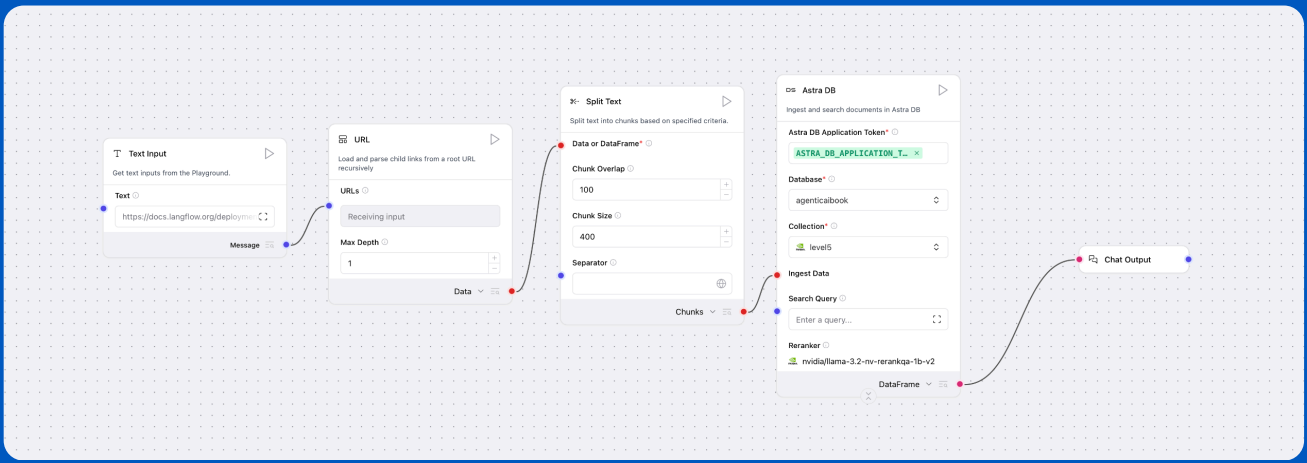
Purpose:

- To process and ingest documents (from URLs or text) into a vector database for later retrieval.

Key Components:

- **TextInput / URLComponent:** Accepts raw text or URLs to fetch content.
- **SplitText:** Splits large documents into manageable chunks for embedding.
- **AstraDB:** Stores the vectorized document chunks for semantic search.
- **ChatOutput:** Displays confirmation or status messages.

FLOW DIAGRAM



How it works:

1. User provides text or a URL.
2. The content is split into smaller chunks.
3. Each chunk is embedded and stored in Astra DB as a vector.
4. The system confirms successful ingestion.

2. RETRIEVAL PIPELINE

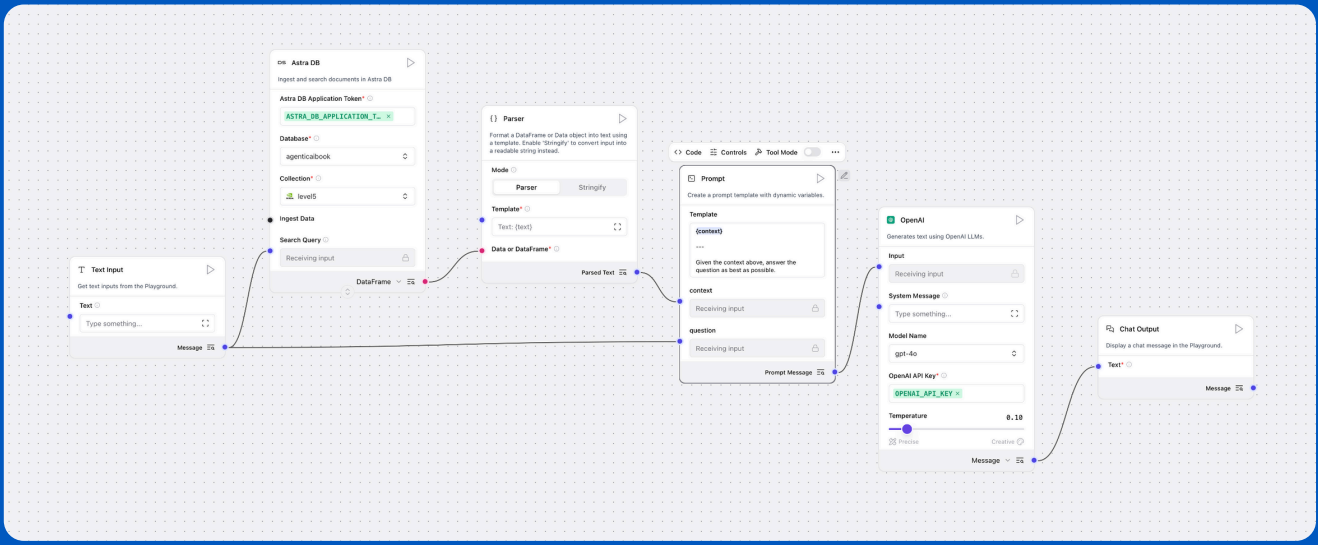
Purpose:

- To answer user questions by retrieving relevant context from the vector store and using an LLM to generate answers.

Key Components:

- **TextInput:** Accepts the user's question.
- **AstraDB:** Performs a similarity search to retrieve relevant document chunks.
- **Parser:** Formats the retrieved context for the prompt.
- **Prompt:** Combines the context and question into a prompt template.
- **OpenAIModel:** Calls the LLM (e.g., GPT-4) to generate an answer.
- **ChatOutput:** Displays the answer to the user.

FLOW DIAGRAM



How it works:

1. User asks a question.
2. The question is sent to Astra DB, which retrieves the most relevant document chunks.
3. The retrieved context is parsed and formatted.
4. A prompt is constructed with both the context and the question.
5. The prompt is sent to the LLM, which generates an answer.
6. The answer is displayed to the user.

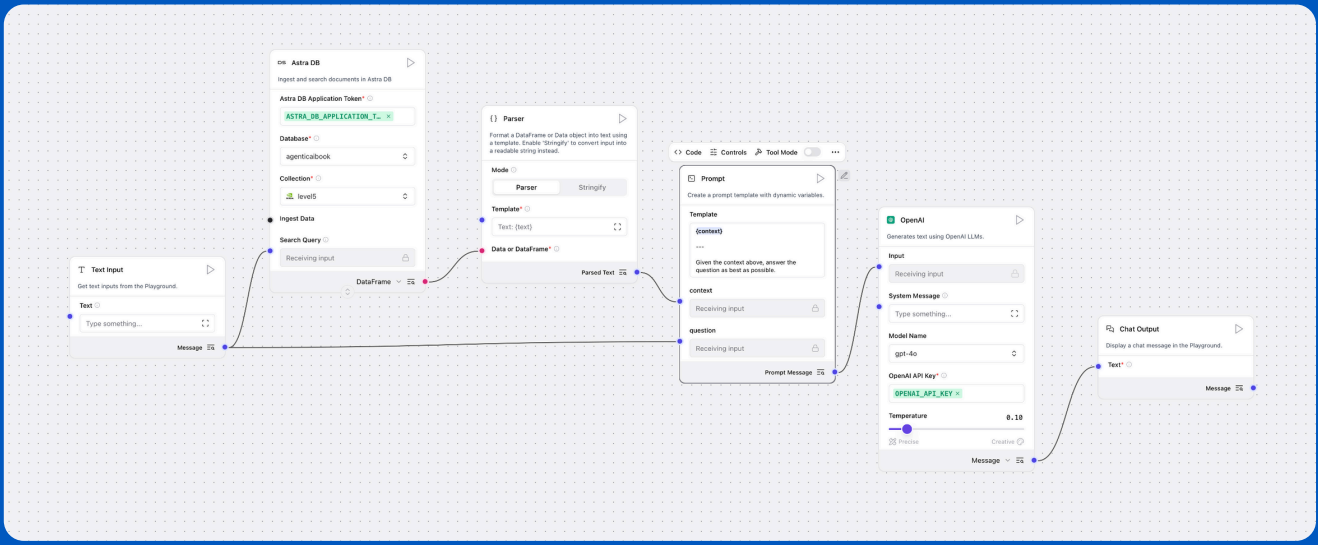
RAG PATTERN OVERVIEW

Retrieval-Augmented Generation (RAG) combines the strengths of information retrieval and generative AI:

- **Retrieval:** Finds relevant information from a large corpus (vector store).
- **Augmentation:** Supplies this information as context to the LLM.
- **Generation:** The LLM uses both the context and the question to generate accurate, grounded answers.

This approach enables the system to answer questions based on up-to-date or domain-specific knowledge, even if the LLM was not trained on that data.

FLOW DIAGRAM



How it works:

1. User asks a question.
2. The question is sent to Astra DB, which retrieves the most relevant document chunks.
3. The retrieved context is parsed and formatted.
4. A prompt is constructed with both the context and the question.
5. The prompt is sent to the LLM, which generates an answer.
6. The answer is displayed to the user.

ADVANCED FLOWS DESIGNED FOR AGENTIC AI

This section contains advanced Langflow flows designed for agentic AI use cases, as part of the Agentic AI Book project. Each flow is modular and can be used as a standalone tool or orchestrated together via the central MainFlow.json agent. Next is an in-depth overview of each flow and how they work together.

MAINFLOW

Purpose: Central agent orchestration. Exposes multiple tool flows as agent tools, allowing the agent to select and use them based on user intent.

→ **Agent Node:** Uses an LLM (default: OpenAI) with a detailed system prompt instructing it to use the right tool for each user request.

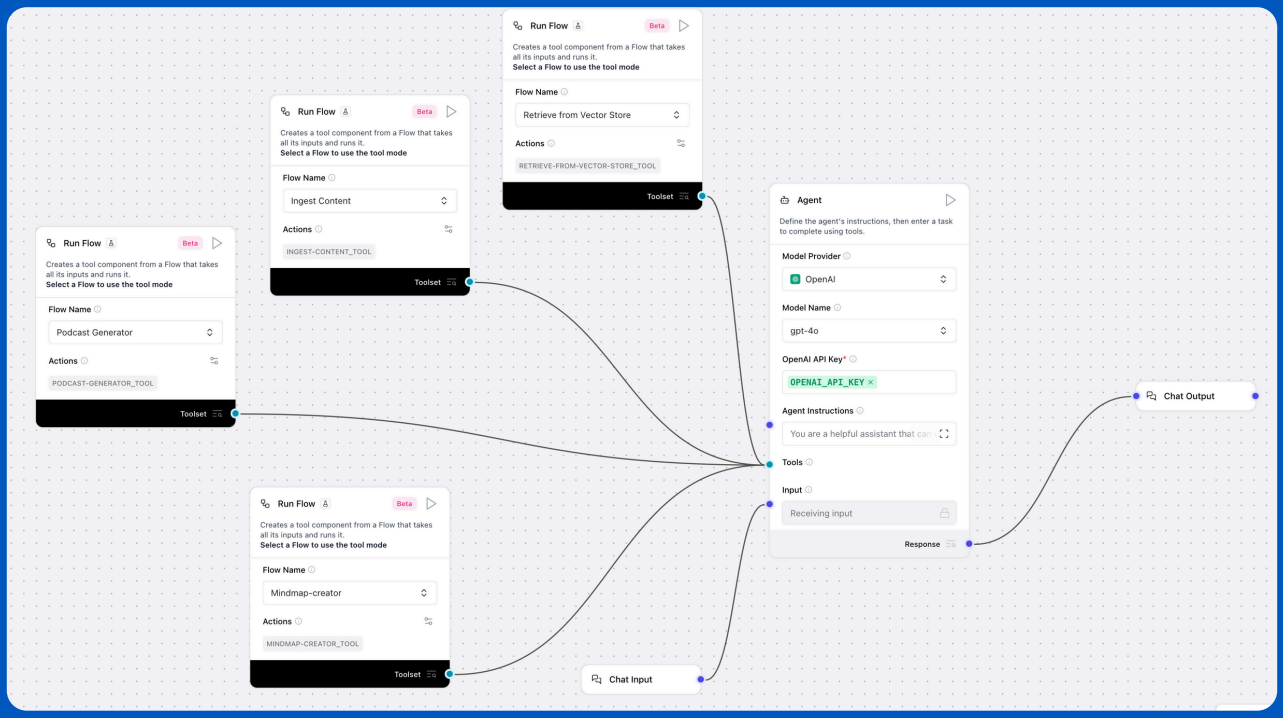
→ **Tools Exposed:**

- Retrieve-from-Vector-Store_tool (RAG Q&A)
- Ingest-Content_tool (webpage ingestion)
- Podcast-Generator_tool (podcast audio generation)
- Mindmap-creator_tool (mindmap creation)

→ **How it works:**

- User input is routed to the agent.
- The agent decides, based on the system prompt and user query, which tool to invoke.
- Each tool is a separate Langflow flow, wired in via RunFlow nodes in tool mode.

Below is a visual overview of the MainFlow orchestration:



RETRIEVE FROM VECTOR STORE

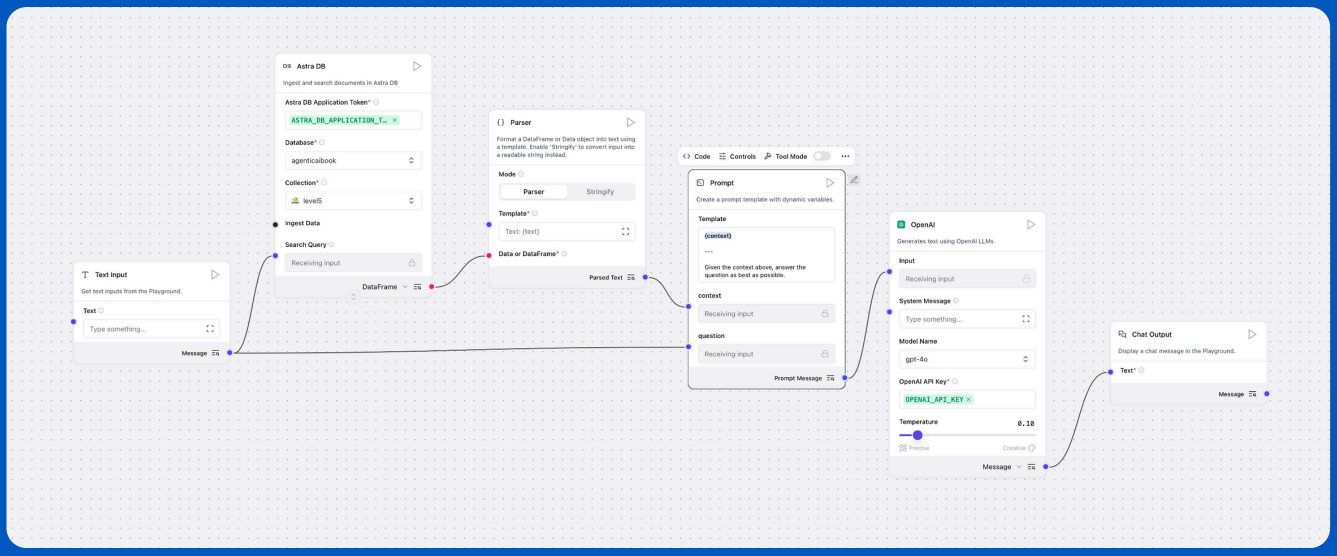
Purpose: Retrieval-Augmented Generation (RAG) Q&A.

→ Pipeline:

- Accepts a user query.
- Uses AstraDB as a vector store to retrieve relevant context.
- Applies a prompt template to combine the query and context.
- Passes the prompt to an OpenAI LLM for answer generation.
- Outputs the answer via a chat output node.

→ Use Case: When the agent needs to answer questions based on ingested or external knowledge.

Below is a visual overview of the Flow :



INGEST CONTENT

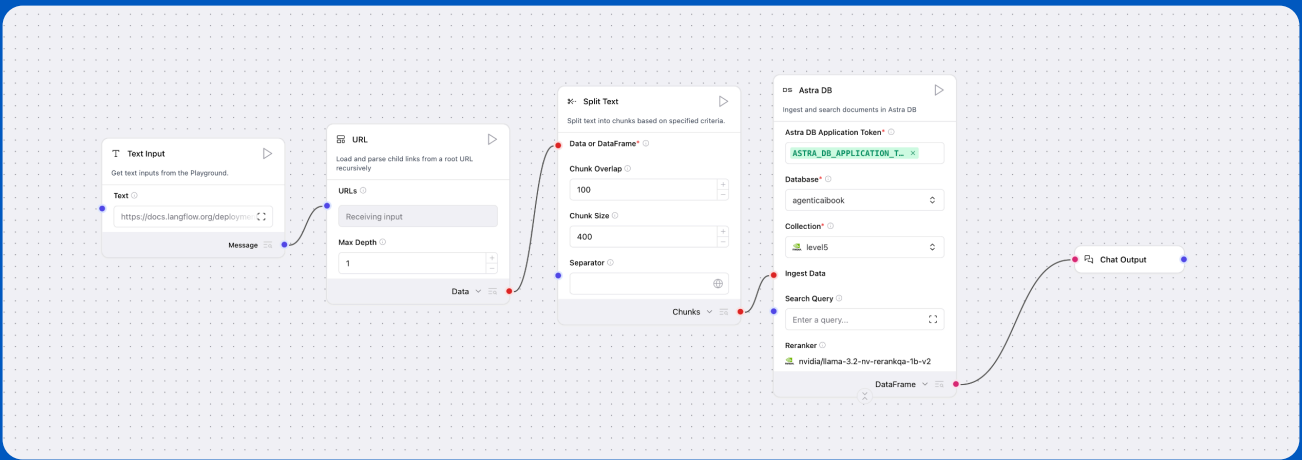
Purpose: Ingests content from a webpage into the vector store for later retrieval.

→ Pipeline:

- Loads content from a user-specified URL.
- Splits the text into manageable chunks.
- Ingests the chunks into AstraDB as vector embeddings.
- Confirms ingestion completion.

→ Use Case: When the agent is asked to "ingest" or "add" new content for future Q&A.

Below is a visual overview of the Flow :



PODCAST GENERATOR

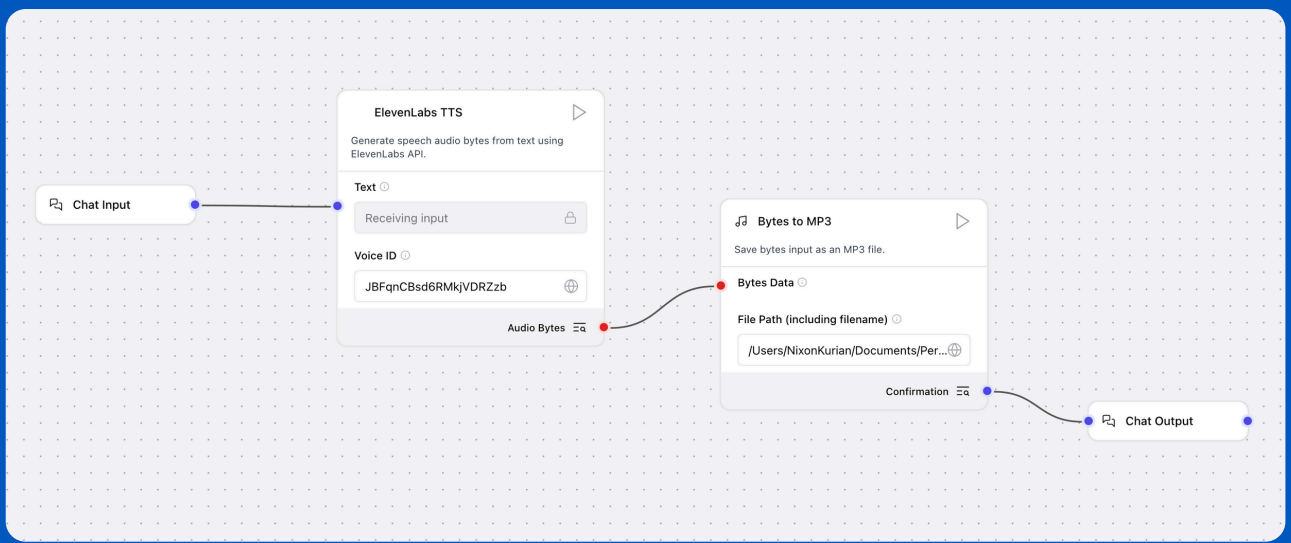
Purpose: Generates a podcast audio file from text.

→ Pipeline:

- Accepts text input.
- Uses ElevenLabs TTS to generate audio bytes. (custom component)
- Saves the audio as an MP3 file. (custom component)
- Outputs a confirmation message with the file location.

→ Use Case: When the agent is asked to "create a podcast" or "generate audio" from text.

Below is a visual overview of the Flow :



MINDMAP-CREATOR

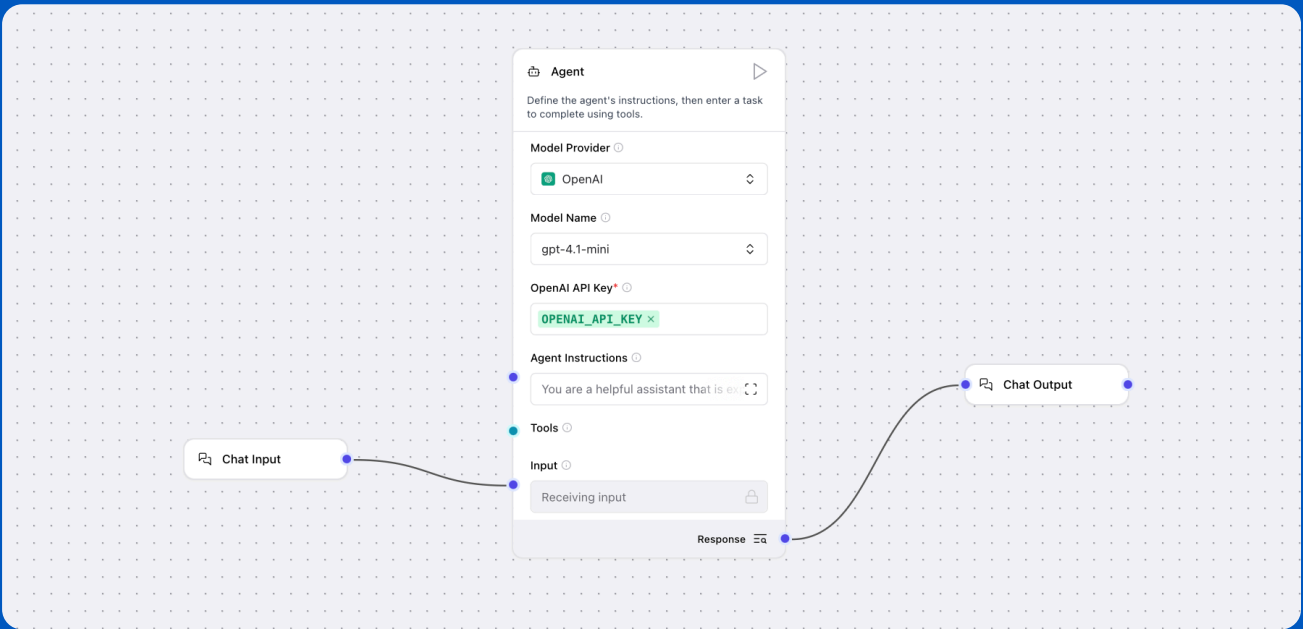
Purpose: Creates a mindmap in mermaid format based on provided context and topic.

→ Pipeline:

- Accepts user input (topic/context).
- Uses an LLM with a system prompt focused on mindmap generation.
- Outputs a mindmap in mermaid syntax.

→ Use Case: When the agent is asked to "create a mindmap" or visualize information.

Below is a visual overview of the Flow :



HOW THESE FLOWS WORK TOGETHER

- All flows are designed to be modular and reusable.
- MainFlow.json acts as the central agent, exposing the other flows as tools.
- The agent's system prompt in MainFlow.json instructs it to use the correct tool based on user intent (e.g., ingestion, Q&A, podcast, mindmap).
- Each tool flow can also be run independently for testing or development.

GETTING STARTED

- Open the desired flow in Langflow Desktop (v1.4.22+ recommended).
- Configure any required API keys (OpenAI, AstraDB, ElevenLabs, etc.) in the agent or tool nodes.
- Use MainFlow.json for a unified agent experience, or run individual flows for specific tasks.

USAGE EXAMPLE

1. Ingest Content

Ingest the following - ***https://cladiusfernando.com/excellence/***

This command instructs the agent to fetch and ingest the content from the provided URL into the vector store for future retrieval.

2. Ask a Question and Generate a Podcast

what is the book Excellence about? create a short podcast for it

The agent will:

- Retrieve relevant information about the book "Excellence" from the ingested content using the RAG flow.
- Generate a podcast audio summary using the Podcast Generator tool.

3. Podcast Output

The generated podcast audio will be saved as an MP3 file.

Interested in more content like this?

Follow me :

OM NALINDE